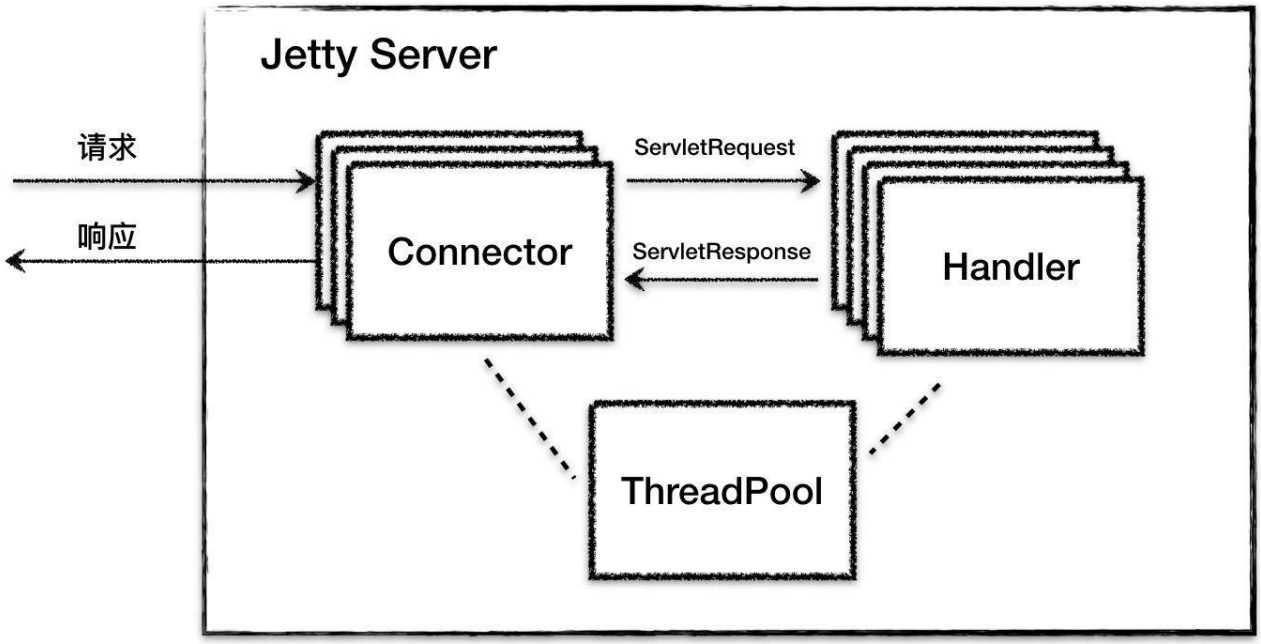
19-比较：Jetty的线程策略EatWhatYouKill

我在前面的专栏里介绍了Jetty的总体架构设计，简单回顾一下，Jetty总体上是由一系列Connector、一系列Handler和一个ThreadPool组成，它们的关系如下图所示：



相比较Tomcat的连接器，Jetty的Connector在设计上有自己的特点。Jetty的Connector支持NIO通信模型，我们知道**NIO模型中的主角就是Selector**，Jetty在Java原生Selector的基础上封装了自己的Selector，叫作ManagedSelector。ManagedSelector在线程策略方面做了大胆尝试，将I/O事件的侦测和处理放到同一个线程来处理，充分利用了CPU缓存并减少了线程上下文切换的开销。

具体的数字是，根据Jetty的官方测试，这种名为“EatWhatYouKill”的线程策略将吞吐量提高了8倍。你一定很好奇它是如何实现的吧，今天我们就来看一看这背后的原理是什么。

Selector编程的一般思路

常规的NIO编程思路是，将I/O事件的侦测和请求的处理分别用不同的线程处理。具体过程是：

启动一个线程，在一个死循环里不断地调用select方法，检测Channel的I/O状态，一旦I/O事件达到，比如数据就绪，就把该I/O事件以及一些数据包装成一个Runnable，将Runnable放到新线程中去处理。

在这个过程中按照职责划分，有两个线程在干活，一个是I/O事件检测线程，另一个是I/O事件处理线程。我们仔细思考一下这两者的关系，其实它们是生产者和消费者的关系。I/O事件侦测线程作为生产者，负责“生产”I/O事件，也就是负责接活儿的老板；I/O处理线程是消费者，它“消费”并处理I/O事件，就是干苦力的员工。把这两个工作用不同的线程来处理，好处是它们互不干扰和阻塞对方。

Jetty中的Selector编程

然而世事无绝对，将I/O事件检测和业务处理这两种工作分开的思路也有缺点。当Selector检测读就绪事件时，数据已经被拷贝到内核中的缓存了，同时CPU的缓存中也有这些数据了，我们知道CPU本身的缓存比内存快多了，这时当应用程序去读取这些数据时，如果用另一个线程去读，很有可能这个读线程使用另一个CPU核，而不是之前那个检测数据就绪的CPU核，这样CPU缓存中的数据就用不上了，并且线程切换也需要开销。

仅供个人学习 请勿传播 用户标记 mannix小童鞋

因此Jetty的Connector做了一个大胆尝试，那就是用**把I/O事件的生产和消费放到同一个线程来处理**，如果这两个任务由同一个线程来执行，如果执行过程中线程不阻塞，操作系统会用同一个CPU核来执行这两个任务，这样就能利用CPU缓存了。那具体是如何做的呢，我们还是来详细分析一下Connector中的ManagedSelector组件。

ManagedSelector

ManagedSelector的本质就是一个Selector，负责I/O事件的检测和分发。为了方便使用，Jetty在Java原生的Selector上做了一些扩展，就变成了ManagedSelector，我们先来看看它有哪些成员变量：

```
public class ManagedSelector extends ContainerLifeCycle implements Dumpable
{
    //原子变量，表明当前的ManagedSelector是否已经启动
    private final AtomicBoolean _started = new AtomicBoolean(false);

    //表明是否阻塞在select调用上
    private boolean _selecting = false;

    //管理器的引用，SelectorManager管理若干ManagedSelector的生命周期
    private final SelectorManager _selectorManager;

    //ManagedSelector不止一个，为它们每人分配一个id
    private final int _id;

    //关键的执行策略，生产者和消费者是否在同一个线程处理由它决定
    private final ExecutionStrategy _strategy;

    //Java原生的Selector
    private Selector _selector;

    //"Selector更新任务"队列
    private Deque<SelectorUpdate> _updates = new ArrayDeque<>();
    private Deque<SelectorUpdate> _updateable = new ArrayDeque<>();

    ...
}
```

这些成员变量中其他的都好理解，就是“Selector更新任务”队列_updates和执行策略_strategy可能不是很直观。

SelectorUpdate接口

为什么需要一个“Selector更新任务”队列呢，对于Selector的用户来说，我们对Selector的操作无非是将Channel注册到Selector或者告诉Selector我对什么I/O事件感兴趣，那么这些操作其实就是对Selector状态的更新，Jetty把这些操作抽象成SelectorUpdate接口。

```
/**
 * A selector update to be done when the selector has been woken.
 */
public interface SelectorUpdate
{
```

```
    void update(Selector selector);
}
```

这意味着如果你不能直接操作ManageSelector中的Selector，而是需要向ManagedSelector提交一个任务类，这个类需要实现SelectorUpdate接口update方法，在update方法里定义你想要对ManagedSelector做的操作。

比如Connector中Endpoint组件对读就绪事件感兴趣，它就向ManagedSelector提交了一个内部任务类ManagedSelector.SelectorUpdate：

```
_selector.submit(_updateKeyAction);
```

这个_updateKeyAction就是一个SelectorUpdate实例，它的update方法实现如下：

```
private final ManagedSelector.SelectorUpdate _updateKeyAction = new ManagedSelector.SelectorUpdate()
{
    @Override
    public void update(Selector selector)
    {
        //这里的updateKey其实就是调用了SelectionKey.interestOps(OP_READ);
        updateKey();
    }
};
```

我们看到在update方法里，调用了SelectionKey类的interestOps方法，传入的参数是OP_READ，意思是现在我对这个Channel上的读就绪事件感兴趣了。

那谁来负责执行这些update方法呢，答案是ManagedSelector自己，它在一个死循环里拉取这些SelectorUpdate任务类逐个执行。

Selectable接口

那I/O事件到达时，ManagedSelector怎么知道应该调哪个函数来处理呢？其实也是通过一个任务类接口，这个接口就是Selectable，它返回一个Runnable，这个Runnable其实就是I/O事件就绪时相应的处理逻辑。

```
public interface Selectable
{
    //当某一个Channel的I/O事件就绪后，ManagedSelector会调用的回调函数
    Runnable onSelected();

    //当所有事件处理完了之后ManagedSelector会调的回调函数，我们先忽略。
    void updateKey();
}
```

仅供个人学习 请勿传播 用户标记 mannix小查理

ManagedSelector在检测到某个Channel上的I/O事件就绪时，也就是说这个Channel被选中了，ManagedSelector调用这个Channel所绑定的附件类的onSelected方法来拿到一个Runnable。

这句话有点绕，其实就是ManagedSelector的使用者，比如Endpoint组件在向ManagedSelector注册读就绪事件时，同时也要告诉ManagedSelector在事件就绪时执行什么任务，具体来说就是传入一个附件类，这个附件类需要实现Selectable接口。ManagedSelector通过调用这个onSelected拿到一个Runnable，然后把Runnable扔给线程池去执行。

那Endpoint的onSelected是如何实现的呢？

```
@Override
public Runnable onSelected()
{
    int readyOps = _key.readyOps();

    boolean fillable = (readyOps & SelectionKey.OP_READ) != 0;
    boolean flushable = (readyOps & SelectionKey.OP_WRITE) != 0;

    // return task to complete the job
    Runnable task= fillable
            ? (flushable
                    ? _runCompleteWriteFillable
                    : _runFillable)
            : (flushable
                    ? _runCompleteWrite
                    : null);

    return task;
}
```

上面的代码逻辑很简单，就是读事件到了就读，写事件到了就写。

ExecutionStrategy

铺垫了这么多，终于要上主菜了。前面我主要介绍了ManagedSelector的使用者如何跟ManagedSelector交互，也就是如何注册Channel以及I/O事件，提供什么样的处理类来处理I/O事件，接下来我们来看看ManagedSelector是如何统一管理和维护用户注册的Channel集合。再回到今天开始的讨论，ManagedSelector将I/O事件的生产和消费看作是生产者消费者模式，为了充分利用CPU缓存，生产和消费尽量放到同一个线程处理，那这是如何实现的呢？Jetty定义了ExecutionStrategy接口：

```
public interface ExecutionStrategy
{
    //只在HTTP2中用到，简单起见，我们先忽略这个方法。
    public void dispatch();

    //实现具体执行策略，任务生产出来后可能由当前线程执行，也可能由新线程来执行
    public void produce();

    //任务的生产委托给Producer内部接口，
```

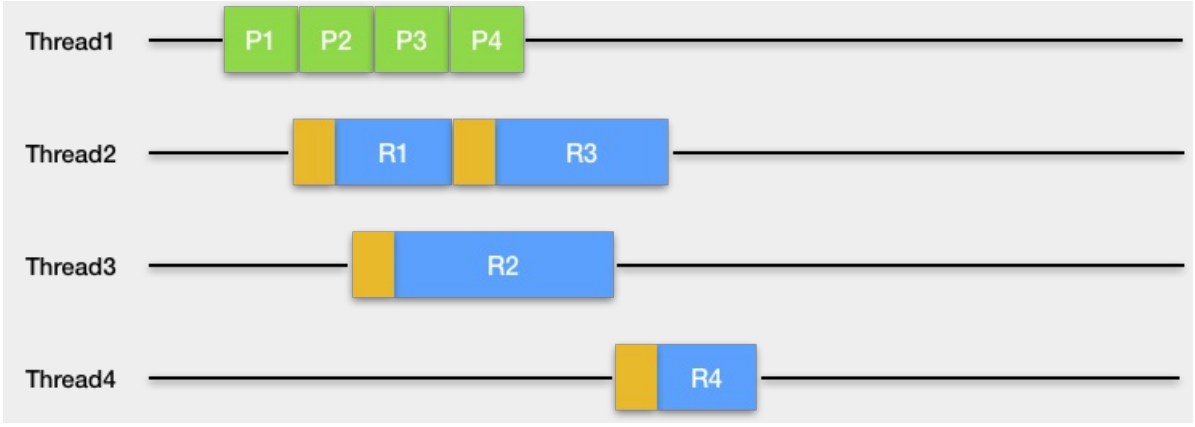
```
    public interface Producer
    {
        //生产一个Runnable(任务)
        Runnable produce();
    }
}
```

我们看到ExecutionStrategy接口比较简单，它将具体任务的生产委托内部接口Producer，而在自己的produce方法里来实现具体执行逻辑，**也就是生产出来的任务要么由当前线程执行，要么放到新线程中执行**。Jetty提供了一些具体策略实现类：ProduceConsume、ProduceExecuteConsume、ExecuteProduceConsume和EatWhatYouKill。它们的区别是：

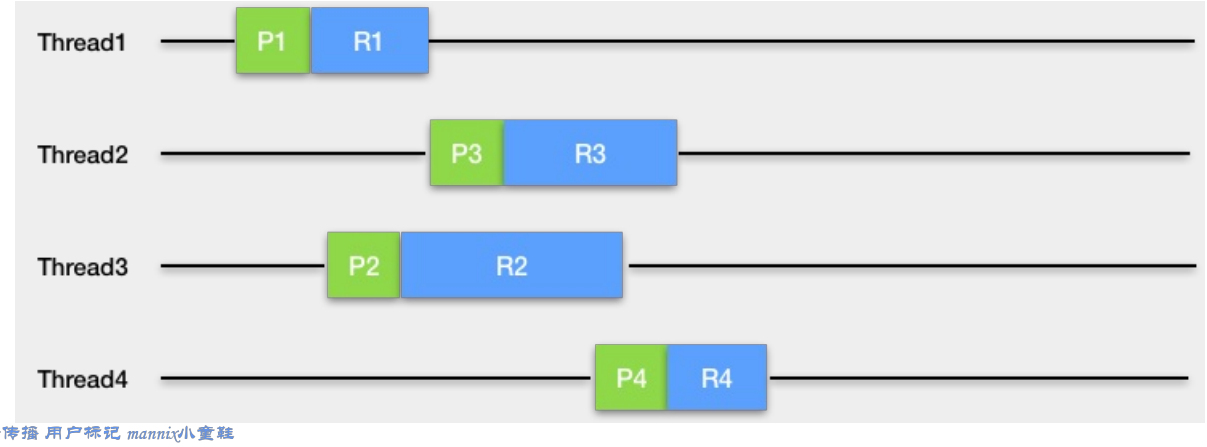
- ProduceConsume：任务生产者自己依次生产和执行任务，对应到NIO通信模型就是用一个线程来侦测和处理一个ManagedSelector上所有的I/O事件，后面的I/O事件要等待前面的I/O事件处理完，效率明显不高。通过图来理解，图中绿色表示生产一个任务，蓝色表示执行这个任务。



- ProduceExecuteConsume：任务生产者开启新线程来运行任务，这是典型的I/O事件侦测和处理用不同的线程来处理，缺点是不能利用CPU缓存，并且线程切换成本高。同样我们通过一张图来理解，图中的棕色表示线程切换。



- ExecuteProduceConsume：任务生产者自己运行任务，但是该策略可能会新建一个新线程以继续生产和执行任务。这种策略也被称为“吃掉你杀的猎物”，它来自狩猎伦理，认为一个人不应该杀死他不吃掉的东西，对应线程来说，不应该生成自己不打算运行的任务。它的优点是能利用CPU缓存，但是潜在的问题是如果处理I/O事件的业务代码执行时间过长，会导致线程大量阻塞和线程饥饿。



仅供个人学习 请勿传播 用户标记 mannix小食鞋

- EatWhatYouKill：这是Jetty对ExecuteProduceConsume策略的改良，在线程池线程充足的情况下等同于ExecuteProduceConsume；当系统比较忙线程不够时，切换成ProduceExecuteConsume策略。为什么要这么做呢，原因是ExecuteProduceConsume是在同一线程执行I/O事件的生产和消费，它使用的线程来自Jetty全局的线程池，这些线程有可能被业务代码阻塞，如果阻塞得多了，全局线程池中的线程自然就不够用了，最坏的情况是连I/O事件的侦测都没有线程可用了，会导致Connector拒绝浏览器请求。于是Jetty做了一个优化，在低线程情况下，就执行ProduceExecuteConsume策略，I/O侦测用专门的线程处理，I/O事件的处理扔给线程池处理，其实就是放到线程池的队列里慢慢处理。

分析了这几种线程策略，我们再来看看Jetty是如何实现ExecutionStrategy接口的。答案其实就是实现produce接口生产任务，一旦任务生产出来，ExecutionStrategy会负责执行这个任务。

```
private class SelectorProducer implements ExecutionStrategy.Producer
{
    private Set<SelectionKey> _keys = Collections.emptySet();
    private Iterator<SelectionKey> _cursor = Collections.emptyIterator();

    @Override
    public Runnable produce()
    {
        while (true)
        {
            //如何Channel集合中有I/O事件就绪，调用前面提到的Selectable接口获取Runnable,直接返回给ExecutionStrategy去
            Runnable task = processSelected();
            if (task != null)
                return task;

           //如果没有I/O事件就绪，就干点杂活，看看有没有客户提交了更新Selector的任务，就是上面提到的SelectorUpdate任务类。
            processUpdates();
            updateKeys();

           //继续执行select方法，侦测I/O就绪事件
            if (!select())
                return null;
        }
    }
 }
```

SelectorProducer是ManagedSelector的内部类，SelectorProducer实现了ExecutionStrategy中的Producer接口中的produce方法，需要向ExecutionStrategy返回一个Runnable。在这个方法里SelectorProducer主要干了三件事情

1. 如果Channel集合中有I/O事件就绪，调用前面提到的Selectable接口获取Runnable，直接返回给ExecutionStrategy去处理。
2. 如果没有I/O事件就绪，就干点杂活，看看有没有客户提交了更新Selector上事件注册的任务，也就是上面提到的SelectorUpdate任务类。
3. 干完杂活继续执行select方法，侦测I/O就绪事件。

本期精华

多线程虽然是提高并发的法宝，但并不是说线程越多越好，CPU缓存以及线程上下文切换的开销也是需要考虑的。Jetty巧妙设计了EatWhatYouKill的线程策略，尽量用同一个线程侦测I/O事件和处理I/O事件，充分利

用了CPU缓存，并减少了线程切换的开销。

课后思考

文章提到ManagedSelector的使用者不能直接向它注册I/O事件，而是需要向ManagedSelector提交一个SelectorUpdate事件，ManagedSelector将这些事件Queue起来由自己来统一处理，这样做有什么好处呢？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- nightmare 2019-06-22 16:45:59
  第一selectorUpdate可以统一抽象封装注册的io事件，坐到面相抽象编程，将来如果nio的api接口有变动，也不需要改动ManangerSelector的代码 只需要新建一下selectorUpdate的子类来实现变更 第二 作为缓冲，，如果高并发的话，一下子很多channel注册到linux的epoll模型上，红黑树的层级就会很大，select的时候就比较耗时，有一个缓冲，可以均匀注册到linux的epoll模型上，既不会快速让红黑树发生过多旋转，也不会过多占用太多文件描述符 [3赞]

- QQ怪 2019-06-23 16:14:40
  课后问题就有点像分布式服务为什么爱用消息中间件一样，一切都是为了解耦，服务类比线程，服务与服务可以直接通讯，线程与线程也可以直接通讯，服务有时候会比较忙或者挂掉了，会导致该请求消息丢失，线程与线程之间的上下文切换同样会带来很大的性能消耗，如果此时的线程池没有多余的线程使用，可能会导致线程积压致使cpu飙高，使用队列能够有效平衡消费者和生产者之间的压力，达到高效率执行。[1赞]

  作者回复2019-06-24 20:11:57
  是的，还有个原因是各个Process线程直接改变Poller的状态需要加锁，将Update的操作封装成任务放到队列串行执行无需加锁

- magicnum 2019-06-22 11:40:08
  不直接注册I/O事件是为了解耦吧，而且队列平衡了生产者消费者的处理能力 [1赞]

仅供个人学习 请勿传播 用户标记 mannix小童鞋

- -W.LI- 2019-06-24 06:21:41
  李老师!SelectorUpdate接口的update方法有个入参完了。硬是没看见哪用了这个入参。泪流满面老师。

- -W.LI- 2019-06-24 06:10:16
  李老师好!越看越不明白了，我承认我基础差也没去看源码。不晓得别的同学看不看的懂。
  老师能不能画点图方便理解。managedSelector怎么工作的还是不清楚。
  只看懂了四种生成消费模型
  1.单线程
  2.io多路复用，把selector当做生成着，处理的业务逻辑单做消费者。(缺点就是不能用缓存，接受任务和处理任务不在一个线程)
  3.就是接受和处理放一个线程，整体放在多线程里面跑。好处就是可以用缓存
  4.对3和2的优化。空余线程多的时候用3，不足用2。把接受到的任务缓存起来防止拒绝太多任务。
  后面的就看不懂了。
  问题1.managedSelector只负责监听read事件么?。accept事件他监听么?一个端口能被多个serversocketchannel监听么?
  问题2.文中说managedselector由endpoint管理，具体怎么管理的呢?managedselector有多少个?怎么创建的?不同的managedselector之间channel共享么(个人感觉不共享会有并发冲突)?
  问题3.线程不够用的时候具体怎么切换的呢?慢慢的全部都转成2还是保留一部分线程执行3一部分执行2。
  问题4.我连channel是怎么注册到managedselector上的都没看明白。更别说销毁了。
  其实就是想知道这几个类的生命周期，然候就是直接执行交互图，纯文字我想不出。希望老师解惑不胜感激

- -W.LI- 2019-06-24 05:08:45
  老师好!我有个问题，我是先把学校教的计算机基础的书操作系统，计算机组成原理这些再看一遍补基础。还是直接上手撸源码啊？

  作者回复2019-06-24 20:19:32
  可以看看操作系统相关的部分，比如进程，虚拟地址，用户空间，内核空间。

- -W.LI- 2019-06-24 05:04:00
  老实好!
  检测到读就绪的时候数据已被拷贝到了内核缓存中。CPU的缓存中也有这些数据。
  这句话怎么理解啊，CPU的缓存说的是高速缓存么?然后内核缓存是什么呢？这方面的知识需要看啥书补啊(是操作系统么?)?IO模型，数据需要拷贝的次数我一点搞不懂

  作者回复2019-06-24 20:07:41
  内核缓存是指，一个进程的虚拟地址地址空间分为用户空间和内核空间，我在22答疑篇会详细说说，这些都是操作系统的知识。

- 802.11 2019-06-23 10:15:46
  老师，在NIO2中服务器端的几个概念不是很清晰，比如WindowsAsynchronousChannelProvider，Invoker等。如果需要系统了解这几个概念是怎么串起来的，怎么办呢。

  作者回复2019-06-24 20:12:24
  建议阅读JVM底层源码